

TravelAgentic

Agentic, budget-first travel planner for college students (U.S. only in V1) — revised team plan

What changed in this revision: DOT Table 6 is now the main flight dataset (BTS DB1C moves to V2, Kaggle is supplemental); flight rollup and booking-curve tables are removed; V1 is **U.S. only**; deployment is on **AWS**; the team is 9 people in 4 sub-teams; search logging is dropped; the app shows real bookable options and links out to book them but never books. Every page ends with a purple **V2** box.

Table of Contents

Page	Section	What you'll find
1	Cover, TOC, Project at a Glance	Goal, scope, team, stack, glossary
2	End-to-End Flow	From “Miami to Tampa” to a list of options with booking links
3	Database	ORM models vs. schemas, tables, indexes, ingestion (vs. the reference project)
4	Flights	DOT Table 6, cleaning, trend queries, one-way vs. round trip, deal labels
5	Hotels	Pricing-data gap, Dotlas, SQL table filtering vs. RAG
6	Duffel API & Routes	Test vs. live, round trips and connections, routes, booking links, cost safety
7	Agentic LLM (LangGraph)	State, nodes, tools, loops, where RAG plugs in
8	Roadmap & Version 2	Build order by sub-team, full V2 list, open items

Project at a Glance

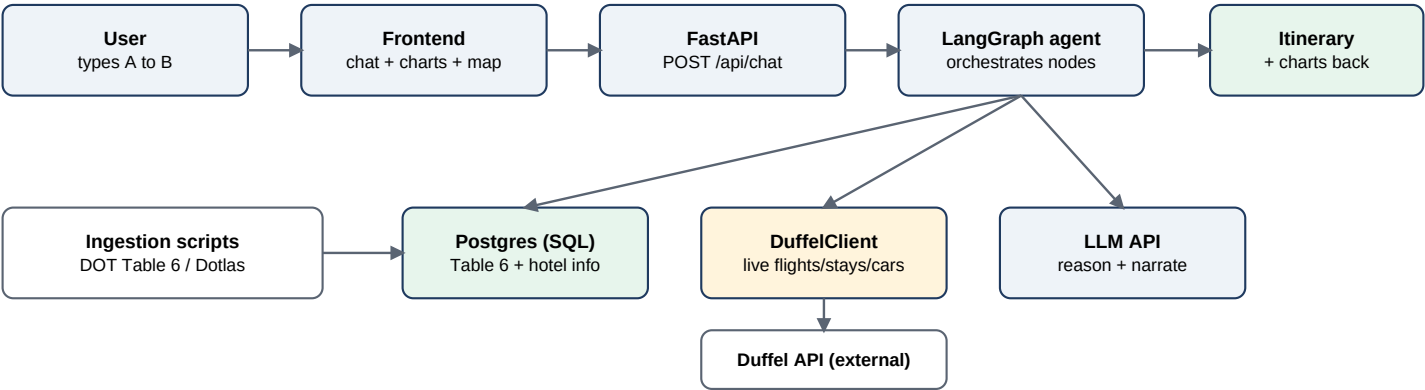
Item	Decision
Goal	User enters an origin and destination; the agent asks follow-ups (fly or drive, one-way or round trip, dates, budget, car, activities) and returns real, currently bookable flights, hotels and cars that fit the budget, plus charts of historically cheaper quarters.
Booking	The app does not book . Each option links out to the airline / hotel / rental site (V1).
Scope	U.S. domestic only in V1 — our historical flight data (DOT Table 6) covers the contiguous U.S.
Team (9)	Flight · Hotel · Routing (FastAPI routes, Duffel client, LangGraph agent) · Frontend
Stack	FastAPI + SQLAlchemy ORM + Pydantic schemas (same as the reference project), PostgreSQL, LangGraph, an LLM API (OpenAI / Groq / Claude), frontend framework TBD. Deployed on AWS .
Data	Flights: DOT Consumer Airfare Report, Table 6 (main), Kaggle flight prices (supplemental). Hotels: Dotlas. Live prices for flights, hotels and cars: Duffel .

Quick Glossary

Term	Meaning in this project
ORM model / schema	Model = Python class defining a table (models/problem.py). Schema = Pydantic class defining what a route returns (schemas/problem.py).
Tool	A Python function the LLM may call (e.g. get_price_trend). The LLM chooses; our code does the work.
Each-way fare	Table 6's average price per direction. A round trip is compared against 2 × this fare.
Slice	One leg in a Duffel search: 1 slice = one-way, 2 = round trip, 3+ = multi-city.
Index	A DB lookup structure (index=True) so filters jump to matching rows instead of scanning all.
RAG	Retrieval-Augmented Generation: fetch relevant text first, then add it to the LLM prompt (V2 here).
Circuit breaker	Code that hard-stops Duffel calls past a limit so a bug can't run up a bill.

2. End-to-End Flow

One request, start to finish — who does what



Step	What happens	Component
1. Ask	User types “Miami to Tampa.” Frontend sends it to POST /api/chat.	Frontend → FastAPI
2. Clarify	Agent fills what it knows and asks for the rest: fly or drive, one-way or round trip, dates or flexibility, budget, travelers, car, activities. Rejects non-U.S. trips politely.	LangGraph intake/clarify
3. Trends	Queries Table 6: cheapest quarter, typical each-way fare, cheapest and largest airline; hotel seasonality from Dotlas. Flags early if the budget is below even the cheapest typical fare.	Postgres (Table 6, Dotlas)
4. Live search	Searches real flights (1 or 2 slices), hotels, and cars through the guarded DuffelClient, for the user's dates or the cheapest window.	Duffel API
5. Preferences	Hotels filtered by city / price / stars in SQL, then ranked by the LLM against “quiet, walkable” (V2: vector search).	SQL filter + LLM
6. Evaluate	Totals vs. budget; labels each flight vs. Table 6 (great / typical / above typical). Over budget → loop back for cheaper dates or relaxed filters.	Evaluator node
7. Respond	Options with prices, stops and duration, a link out to book , route map, cheaper-quarter chart.	Frontend

Three rules every sub-team follows

- **Numbers only come from data.** Every price shown traces back to Duffel or Table 6 / Dotlas. The LLM may say “beach towns cost more in winter,” never invent “\$340/night.”
- **The LLM calls tools; it never writes SQL.** A fixed set of parameterized functions keeps answers correct, reproducible, and safe from SQL injection.
- **Routes return finished answers** ({typical_fare, cheapest_quarter, cheapest_carrier}), not hundreds of raw rows the agent would have to process itself.

Compare with the reference project: its flow is Frontend → routes/problem.py → SQLAlchemy query → Pydantic response. We keep that exact flow for data routes and add one layer: a LangGraph agent in services/ (empty in the reference — see services/llm_service.py) that decides *which* data functions to call and explains the results.

- V2 — Version 2 additions**
- International trips (needs a non-U.S. historical fare source).
 - In-app booking through Duffel orders instead of links out (real money + compliance; out of scope for V1).

3. Database

PostgreSQL + SQLAlchemy ORM + Pydantic schemas, modeled on the reference project

How it works

Same three layers as the reference project: **models** define storage, **schemas** define what routes return, **scripts** load the datasets once. Models mirror the source data; schemas are shaped for the LLM tools. No pre-aggregated rollup tables: Table 6 is already averaged by the DOT, so routes query it directly with indexes (a single route has ~120 rows across all years).

Reference project file	TravelAgentic equivalent	Notes
db/database.py	db/database.py (reuse)	engine, SessionLocal, get_db()
core/config.py	add DUFFEL_API_KEY, DUFFEL_MODE, LLM key, call limits	fills the # ADD LLM API KEY HERE slot
models/user.py, core/security.py, routes/auth.py	reuse nearly unchanged	pwdlib hashing + JWT login
models/problem.py	models/flights.py, models/hotels.py, models/locations.py	flat tables; no Tag-style many-to-many
schemas/problem.py (Basic vs Detail)	schemas/flights.py, schemas/hotels.py	e.g. HotelBasicResponse vs HotelDetailResponse
scripts/ingest_problems.py	scripts/ingest_table6.py, ingest_hotels.py	pandas chunks + bulk insert
index=True on question_id	index=True on city1, city2, year, quarter	fast route lookups

V1 tables

Table	Holds	Fed by
users	account info, hashed password	reference project
flight_fares	year, quarter, city1, city2, nsmiles, passengers, fare, carrier_lg, large_ms, fare_lg, carrier_low, lf_ms, fare_low (each-way fares)	DOT Table 6
locations (shared lookup)	Table 6 city name ("Miami, FL (Metropolitan Area)"), state, airport codes (MIA, FLL), hotel city name	small hand-built table
hotels	name, city, lat/lng, stars, review score, amenities, policies, description (U.S. rows only)	Dotlas hotels
hotel_rate_calendar	hotel/city, check-in date, quoted nightly rate (future quotes)	Dotlas pricing
api_usage	daily and per-minute Duffel call counters	DuffelClient

Roadblock	Solution
Do flights and hotels need a JOIN?	Not directly — they're unrelated facts. Both point to the shared locations table, which also translates the user's "Miami" into Table 6's market name and Duffel's airport codes.
Table 6 stores numbers as text	Fares look like \$173.72 and years like 2,023. Strip \$ and commas and cast to numbers during ingestion.
Useless columns	Drop the 13 empty Geocoded_* columns, tbl, table_1_flag.
Schema changes mid-semester	create_tables() works at first; adopt Alembic migrations once 9 people share one AWS database.

V2 — Version 2 additions

- BTS DB1C monthly ticket data + Postgres **materialized views** (stored GROUP BY results) once raw data is millions of rows.
- saved_trips per user (mirrors the reference user_favorites table) and price alerts.
- pgvector extension in the same Postgres for hotel description embeddings (no separate vector DB).

4. Flights

DOT Table 6 for history and baselines; Duffel for what is bookable now

How it works

Table 6 answers *which quarter is usually cheaper, which airline is usually cheapest, and what a typical fare is*. Duffel answers *what can be booked for these dates*. The Flight team owns ingesting and cleaning Table 6 and the trend functions the agent calls.

Dataset	Role	Notes
DOT Consumer Airfare Report, Table 6	Main flight table	Quarterly, 1990s to Q1 2026; every contiguous-U.S. city pair averaging 10+ passengers/day. Average each-way fare paid, includes connecting trips, both directions combined. Carrier columns: largest airline and lowest-fare airline with their fares.
Kaggle dilwong/flightprices	Supplemental	Expedia itinerary-level search results (Apr–Oct 2022). Use as a realistic template for hand-written mock Duffel offers and to sanity-check Table 6 baselines. Not loaded into production tables.

Example from our own query (Miami ↔ Tampa, 2023 – Q1 2026)

Quarter	Avg each-way fare	Largest airline (AA)	Lowest-fare airline (NK)
Q1 (Jan–Mar)	\$197	\$263	\$98
Q2 (Apr–Jun)	\$178	\$232	\$92
Q3 (Jul–Sep)	\$174	\$239	\$90
Q4 (Oct–Dec)	\$183	\$250	\$94

The agent learns: Q3 is cheapest, Q1 runs ~13% higher, Spirit is reliably cheapest (~\$90), American dominates (~\$250). A \$60 budget is flagged before any Duffel call; a \$240 AA result is labeled “typical.”

Trend tools exposed to the agent (query Table 6 directly, recent years only, default 2023+)

Tool	Returns
get_price_trend(origin_city, dest_city)	typical each-way fare, by-quarter averages, years of data
get_cheapest_quarters(origin_city, dest_city)	4 quarters ranked → feeds the “cheaper times to travel” chart
get_carrier_baseline(origin_city, dest_city)	largest + lowest-fare airline, their fares and market shares

Deal labels (rule-based in V1): compare each Duffel offer to the Table 6 fare for that quarter (× 2 for round trips). Roughly 20%+ below = great, within ±20% = typical, above = above typical. No custom ML model is needed for V1.

Roadblock	Solution
Quarterly, not monthly	Say “July–September is usually cheapest,” not “February.” Monthly detail is V2 (DB1C).
City pairs, not airports; one row for both directions	Use <code>locations</code> to map cities ↔ airports; search Duffel on all airports for that city (e.g. MIA + FLL).
Route missing (small market, Hawaii/Alaska)	Return “no history” and rely on Duffel prices only; never guess.
Test mode shows only “Duffel Airways”	Test the evaluator with mock offers modeled on the Kaggle data (page 6).

V2 — Version 2 additions

- **BTS DB1C** (monthly, airport-level, every carrier) for month-level charts and directional routes.
- Regression model of expected fare + residual-based deal classifier; quantile price ranges.
- “When to book” (booking-curve) advice using the Kaggle data, if the app's scope expands to that.

5. Hotels

Static hotel info in our DB + live prices from Duffel Stays

How it works

Hotels split into two halves. **Qualitative** data (name, amenities, stars, description) lives in our `hotels` table. **Price and availability** for the user's dates come live from Duffel Stays. Dotlas future quotes give rough seasonality (which months tend to be pricier) for the chart.

Source	Use it for	Caveat
Dotlas (Databricks Marketplace, free sample)	hotels: amenities, policies, ratings, descriptions. hotel_pricing: quoted rates up to a year ahead → seasonality	Forward-looking, not historical. Sample only — check U.S. city coverage with <code>SELECT COUNT(*)</code> . Needs a free Databricks workspace; export via notebook.
Duffel Stays (live)	Real rates for a date range and radius	Check-in up to 330 days ahead, stays up to 99 nights
ICDM 2013 Expedia (Kaggle)	Optional: testing ingestion code at scale	2012–13 prices; never shown to users

How the agent finds cheap hotels: one Duffel Stays search with a radius covering the whole city returns every available property with an accurate cheapest rate, so we sort by price instead of guessing neighborhoods (guessing is how you miss the true cheapest). The LLM's general knowledge is used only to **explain** results ("this area usually runs cheaper than downtown").

SQL table filtering (V1) vs. RAG (V2)

V1 — SQL filter, then the LLM ranks

Tool `get_hotels_in_city(city, min_stars, amenities)` runs a plain `WHERE` filter, the same pattern as the reference `routes/problem.py /list`. If the result is a few hundred hotels or fewer, pass names + short descriptions into the prompt and let the LLM judge "quiet, walkable, good for 4 people." No embeddings needed.

V2 — add RAG on top

Only needed if filtered results get too large for the prompt or review text is added. Embed descriptions (pgvector in the same Postgres, or Chroma), then: **metadata filter first** (city, price band, stars) → **similarity search** on the vibe → top 10 to the LLM. Filtering first avoids a "perfect quiet hotel" in the wrong city.

Roadblock	Solution
No large, free, current hotel <i>price history</i> exists	Confirmed gap. V1 uses Dotlas for seasonality and Duffel for real prices. Charts are labeled "based on quoted future rates."
Duffel hotel names may not match Dotlas names	Match on name + coordinates (within ~100 m); unmatched hotels still show with Duffel's own name, rating and photos.
Scraping Booking.com / Google Hotels	Ruled out: against their terms and fragile; Booking.com itself lost a 2024 CFAA jury verdict for scraping Ryanair.
Thin Duffel coverage in a city	Hotelbeds has a free test environment as a backup inventory source.

V2 — Version 2 additions

- RAG over hotel descriptions / reviews (metadata filter + similarity search).
- Hotelbeds as a second live inventory source; review-sentiment summaries with an off-the-shelf model.

6. Duffel API & Routes

One vendor for flights, hotels and cars — behind one guarded client

Product	Endpoint	Key fields returned
Flights	POST /air/offer_requests	carrier (name, IATA, logo), total_amount, cabin_class, per-slice segments (→ stops), duration, full airport names, baggage, expires_at (~30 min)
Hotels	POST /stays/search	name, rating, review score, photos, location, cheapest_rate_total_amount, room rates, tax, cancellation timeline
Cars	Duffel Cars	40+ brands. Request access in the dashboard. Needs driver age + country. Test supplier “Duffel Test Drive” gives repeatable success / decline / expired cases.
Airlines	GET /air/airlines	real names, IATA codes, logos → UI filters

Round trips and connections: a search is a list of **slices** (1 = one-way, 2 = round trip). A round-trip offer covers both slices and total_amount is the whole trip. Nonstop and connecting options come back automatically; segments per slice minus 1 = stops. max_connections: 0 = nonstop only (set when the user says “no layovers”), default 1.

Account status: verified as a sole proprietorship under Arjun's name; live mode unlocked. Searching never requires a booking and we never top up the balance.

Test mode vs. live mode

- **Test (free, unlimited):** fake “Duffel Airways” (ZZ) and “Duffel Test Hotel.” Good for plumbing, useless for judging prices → unit-test the evaluator with hand-written mock offers (20% below / at / above the Table 6 fare).
- **Live:** same code, only the key changes (DUFFEL_MODE=test | live). Short, scoped burst for final checks and the demo, then switch back.

Booking links (the app never books)

Duffel offers are real and bookable, but an offer ID can only be booked through Duffel and expires in ~30 minutes, so it can't be turned into a link. V1 shows each option with a link to the **airline's / hotel's / rental company's own site** (from carrier or property name) plus the details to find it (flight number, date). Note in the UI: “price may change at booking.”

Routes (FastAPI, same style as routes/problem.py, prefix /api)

Route	Backed by	Response schema
POST /auth/register, /auth/login	reference routes/auth.py	UserResponse, token
GET /flights/trend, /flights/cheapest-quarters, /flights/carriers	flight_fares (Table 6)	FlightTrend, QuarterRank, CarrierBaseline
GET /hotels/city, /hotels/{id}, /hotels/seasonality	hotels, hotel_rate_calendar	HotelBasic / HotelDetail / Seasonality
POST /live/flights, /live/hotels, /live/cars	DuffelClient → Duffel	normalized offers + booking link
POST /chat	LangGraph agent	message + itinerary + chart data

Routes are thin wrappers around functions in services/; LangGraph tools call the same functions directly, so logic exists once and the agent never calls its own server over HTTP.

Cost safety — DuffelClient circuit breaker

```
# services/duffel_client.py -- the ONLY file allowed to talk to Duffel
def search(path, body):
    calls = db.execute("""INSERT INTO api_usage(day, calls) VALUES (CURRENT_DATE, 1)
        ON CONFLICT (day) DO UPDATE SET calls = api_usage.calls + 1 RETURNING calls""") # atomic
    if calls > settings.DUFFEL_DAILY_LIMIT or per_minute_calls() > settings.DUFFEL_MINUTE_LIMIT:
        raise UsageLimitExceeded() # hard stop; retries (max 3, with backoff) count too
    resp = httpx.post(BASE + path, json=body, headers=auth_headers(), timeout=30)
    return normalize(resp.json()) # keep only the fields our schemas use
```

Plus a virtual card with a hard monthly cap on the account. A possible ~\$0.005 excess-search fee (third-party source, **unverified**) — confirm in the dashboard's billing page.

V2 — Version 2 additions

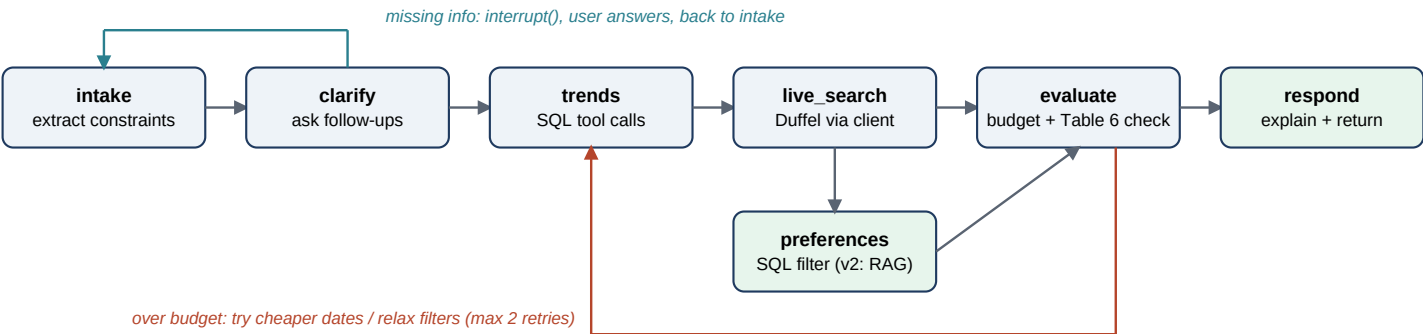
- Duffel **Partial Offer Requests**: pick the outbound flight, then see matching returns (Duffel's recommended search UX).
- In-app booking via Duffel orders; multi-city trips (3+ slices).

7. Agentic LLM (LangGraph)

How the agent connects the database, routes, Duffel and (in V2) RAG

How it works

LangGraph models the conversation as a **graph**: each **node** is a Python function that reads and updates a shared **state**; **edges** pick the next node, and **conditional edges** allow loops (ask again, retry cheaper dates). The LLM extracts constraints, chooses tools and writes the explanation. All numbers come from tools. The Routing team owns this page.



```
class TripState(TypedDict):
    messages: list          # chat history
    constraints: dict        # origin, destination (U.S.), trip_type, depart_date, return_date / flexibility,
                             # budget, travelers, mode (fly/drive), needs_car, max_connections, activities
    missing: list[str]      # fields still needed -> drives the clarify loop
    trends: dict            # Table 6 trend / cheapest quarters / carrier baseline + hotel seasonality
    offers: dict            # normalized Duffel flights / stays / cars (+ booking links)
    hotel_matches: list     # preference-ranked hotels
    evaluation: dict        # totals, budget pass/fail, deal labels, retry count
```

Node	Does	Uses
intake	Turns free text into constraints via structured output; lists missing fields; rejects non-U.S. routes	LLM
clarify	Asks one follow-up at a time; pauses with interrupt() until the user answers	LLM + human-in-the-loop
trends	Calls Table 6 + hotel seasonality tools; picks candidate dates if the user is flexible	SQL tools (pages 4–5)
live_search	1 or 2 slices for flights, plus hotels and cars; skips flights if the user drives	DuffelClient
preferences	SQL-filters hotels, LLM ranks against the user’s vibe (V2: RAG retrieval)	SQL + LLM
evaluate	Totals vs. budget; deal labels vs. Table 6 (×2 for round trips); over budget → back to trends (max 2)	Python
respond	Explanation + itinerary + chart data + map coordinates + booking links	LLM

Roadblock	Solution
LLM invents prices or SQL	Tools only; evaluate rejects any price not present in offers.
Infinite loops / runaway calls	Retry counter in state + LangGraph recursion limit + DuffelClient limits.
Remembering the chat between requests	LangGraph checkpointer keyed by a thread_id per conversation.
Agent work blocked until DB is done	Build nodes now against mocked tools with the agreed signatures; swap in real ones later.

- V2 — Version 2 additions
- RAG retrieval inside the preferences node; Postgres-backed checkpointer so chats survive server restarts.
 - A separate cars/activities node with live Duffel Cars and local-activity suggestions.

8. Roadmap & Version 2

Build order by sub-team, everything deferred to V2, and open items

V1 build order

Phase	Work	Team	Done when
1	Repo skeleton from the reference: db/, core/, auth, config with Duffel / LLM keys; AWS dev database	Routing	register / login works on AWS
2	models/flights.py + locations; Table 6 ingestion and cleaning; trend functions	Flight	Miami–Tampa numbers match page 4
2	models/hotels.py; Dotlas export (U.S. rows), ingestion, seasonality function	Hotel	row counts verified per demo city
3	Pydantic schemas + trend/hotel routes, tested with plain HTTP (no LLM yet)	Routing	3 routes checked by hand
4	DuffelClient with limits; live routes in test mode; mock offers from Kaggle data	Routing	limit test passes
5	LangGraph nodes (started early against mocks), then wired to real services	Routing	Miami → Tampa end to end
3–6	Chat UI, results cards with booking links, route map, cheaper-quarter chart	Frontend	demo flow works
7	Short live-mode burst for demo routes; deploy on AWS; switch back to test	All	demo shows real options

Complete Version 2 list (only if V1 is finished)

Area	V2 feature	Why it waits
Flights	BTS DB1C monthly data + materialized views	Table 6 covers V1; DB1C is millions of rows of ETL
Flights	Fare regression + deal classifier; quantile ranges	Rule-based Table 6 labels are enough for V1
Flights	"When to book" advice (Kaggle booking curve)	Not part of V1's scope
Hotels	RAG over descriptions / reviews; Hotelbeds backup	SQL filter + LLM ranking works at our size
Duffel	Partial Offer Requests, multi-city, in-app booking	V1 links out to book; booking needs money + compliance
Cars	Fully live Duffel Cars in every itinerary	Needs access approval first
Scope	International trips	No non-U.S. fare history yet
Users	Saved trips, price-drop alerts, Postgres-backed chat memory	Needs scheduled jobs and more AWS setup
Original idea	"Cancellation bet" backup plans	Needs cancellation data we don't have

Open items

- Pick 3–5 U.S. demo routes (e.g., Miami ↔ Tampa, Chicago ↔ NYC) and make data quality perfect there first.
- Confirm Duffel's per-search fee (if any) in billing; set daily / per-minute limits; request Duffel Cars access.
- Choose the frontend framework, LLM provider, and AWS services (database, app hosting, secrets for API keys).
- Add teammates under Duffel **Team members** so nobody shares one login.

Next step for every sub-team: read the reference files named on your page, then post your proposed model columns or tool signatures in the shared doc so the Routing team can mock them right away.